

### PREFACE FOR FACULTY

The Parallel Realization of an IIR filter is obtained by performing Partial Fraction Expansion (PFE) on  $H(z)$ . In contrast to Cascade realization where sections are multiplied in series, Parallel realization operates all 1st-order and 2nd-order sections concurrently, summing their outputs.

#### Pedagogical Strategy:

1. Derive the partial fraction expansion:  $H(z) = C + \sum H_k(z)$ .
2. Group complex conjugate pole pairs into real 2nd-order sections:  $\frac{\gamma_{0k} + \gamma_{1k}z^{-1}}{1 + a_{1k}z^{-1} + a_{2k}z^{-2}}$ .
3. Contrast Cascade vs. Parallel realizations:
  - Noise propagation: In Parallel form, roundoff noise from each section goes directly to the output without passing through other poles.
  - Concurrency: Sections execute independently on multi-core / FPGA DSP architectures.
4. Draw complete Parallel signal flow graphs using canonical Direct Form II subsections.

### 1. LEARNING OBJECTIVES

By the end of this lecture, students will be able to:

1. **Perform** Partial Fraction Expansion on high-order rational transfer functions  $H(z)$ .
2. **Combine** complex conjugate residue pairs into real second-order parallel branches.
3. **Draw** complete Parallel Realization signal flow graphs.
4. **Evaluate** noise and latency benefits of parallel architectures.

### 2. MATHEMATICAL FOUNDATIONS

#### 2.1 Parallel Realization Formulation

Given rational  $H(z) = \frac{B(z)}{A(z)}$  with  $M \leq N$ :

$$H(z) = C + \sum_{k=1}^{K_1} \frac{A_k}{1 - p_k z^{-1}} + \sum_{k=1}^{K_2} \frac{\gamma_{0k} + \gamma_{1k} z^{-1}}{1 + a_{1k} z^{-1} + a_{2k} z^{-2}}$$

Where  $C = b_N/a_N$  if  $M = N$  (and  $C = 0$  if  $M < N$ ).

- **Real poles** yield 1st-order sections:  $H_k(z) = \frac{A_k}{1 - p_k z^{-1}}$ .
- **Complex conjugate pole pairs**  $p_k, p_k^*$  yield 2nd-order sections:  $\frac{A_k}{1 - p_k z^{-1}} + \frac{A_k^*}{1 - p_k^* z^{-1}} = \frac{2\text{Re}\{R_k\} - 2\text{Re}\{R_k p_k^*\} z^{-1}}{1 - 2\text{Re}\{p_k\} z^{-1} + |p_k|^2 z^{-2}}$

$$z^{-1} + |p_k|^2 z^{-2} = \frac{\gamma_{0k} + \gamma_{1k} z^{-1}}{1 + a_{1k} z^{-1} + a_{2k} z^{-2}}$$

### 3. WORKED NUMERICAL EXAMPLES

#### Example 19.1: Parallel Realization of a 3rd-Order IIR Filter

**Problem:** Realize  $H(z) = \frac{1+2z^{-1}+z^{-2}}{1-0.75z^{-1}+0.125z^{-2}}$  in Parallel Form.

**Solution:** Since numerator degree  $M = 2$  and denominator degree  $N = 2$ , first divide out constant  $C$ :  
Denominator factors:  $(1 - 0.5z^{-1})(1 - 0.25z^{-1})$ .

$$H(z) = \frac{1 + 2z^{-1} + z^{-2}}{(1 - 0.5z^{-1})(1 - 0.25z^{-1})} = C + \frac{A}{1 - 0.5z^{-1}} + \frac{B}{1 - 0.25z^{-1}}$$

Using polynomial division:  $C = \frac{b_2}{a_2} = \frac{1}{0.125} = 8$ . Alternatively, standard PFE:

$$H(z) = \frac{A_0}{1} + \frac{A_1}{1 - 0.5z^{-1}} + \frac{A_2}{1 - 0.25z^{-1}}$$

Let  $z^{-1} = 2 \Rightarrow A_1 = \frac{1+2(2)+4}{1-0.25(2)} = \frac{9}{0.5} = 18$ . Let  $z^{-1} = 4 \Rightarrow A_2 = \frac{1+2(4)+16}{1-0.5(4)} = \frac{25}{-1} = -25$ . Evaluating at  $z^{-1} = 0$ :  $H(1) = 1 = C + A_1 + A_2 \Rightarrow 1 = C + 18 - 25 \Rightarrow C = 8$ .

$$H(z) = 8 + \frac{18}{1 - 0.5z^{-1}} - \frac{25}{1 - 0.25z^{-1}}$$

### 4. UNIVERSITY EXAMINATION QUESTIONS & MARKING RUBRIC

#### Question 1 (15 Marks)

**(a)** Compare Cascade and Parallel realizations of IIR filters in terms of round-off noise accumulation, hardware pipelining, and pole-zero cancellation. (6 Marks) **(b)** Realize the system function in Parallel Form:

$$H(z) = \frac{1 - z^{-1}}{(1 - 0.5z^{-1})(1 - 0.8z^{-1} + 0.64z^{-2})}$$

Draw the complete SFG. (9 Marks)

#### Model Answer & Step-by-Step Marking Rubric:

- **Part (a):**
  - Comparison table covering noise gain, independent branches, and section latency (6 Marks)
- **Part (b):**
  - Partial fraction setup:  $H(z) = \frac{A}{1-0.5z^{-1}} + \frac{B_0+B_1z^{-1}}{1-0.8z^{-1}+0.64z^{-2}}$  (3 Marks)
  - Residue calculations:  $A = \frac{1-2}{1-0.8(2)+0.64(4)} = \frac{-1}{1-1.6+2.56} = \frac{-1}{1.96} \approx -0.5102$  (3 Marks)
  - Complete Parallel SFG drawn with labeled coefficients and shared input (3 Marks)

## 5. PYTHON VERIFICATION SCRIPT

---

```
import scipy.signal as signal

b = [1, 2, 1]
a = [1, -0.75, 0.125]
r, p, k = signal.residuez(b, a)
print("Residues r:", r)
print("Poles p:", p)
print("Direct term k:", k)
```